

# Contents

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>Les Types en Python</b>                              | <b>4</b>  |
| <b>1 Pourquoi comprendre les types ?</b>                | <b>4</b>  |
| <b>2 Les grandes familles de types</b>                  | <b>4</b>  |
| Types numériques . . . . .                              | 4         |
| Type booléen . . . . .                                  | 4         |
| Types séquentiels . . . . .                             | 4         |
| Types de collection . . . . .                           | 5         |
| <b>3 Les types numériques</b>                           | <b>5</b>  |
| 3.1 Entiers — int . . . . .                             | 5         |
| 3.2 Flottants — float . . . . .                         | 5         |
| 3.3 Complex — complex . . . . .                         | 6         |
| <b>4 Bool — un cas particulier</b>                      | <b>6</b>  |
| <b>5 Les chaînes — str</b>                              | <b>6</b>  |
| <b>6 List — mutable</b>                                 | <b>7</b>  |
| <b>7 Tuple — immuable</b>                               | <b>7</b>  |
| <b>8 Dictionnaire — dict</b>                            | <b>8</b>  |
| <b>9 Set — ensemble unique</b>                          | <b>8</b>  |
| Range — séquence paresseuse                             | 8         |
| <b>1 1 Mutabilité vs Immutabilité</b>                   | <b>9</b>  |
| <b>1 2 Identité vs Égalité</b>                          | <b>9</b>  |
| <b>1 3 Conversion de types</b>                          | <b>10</b> |
| <b>1 4 Typage dynamique</b>                             | <b>10</b> |
| <b>1 5 Typage fort</b>                                  | <b>10</b> |
| <b>1 6 Annotations de type (bonne pratique moderne)</b> | <b>11</b> |
| <b>1 7 Vérification de type</b>                         | <b>11</b> |
| <b>1 8 Copie superficielle et copie profonde</b>        | <b>11</b> |
| <b>1 9 Cas réel : bug de mutabilité</b>                 | <b>12</b> |

|            |                                                                            |           |
|------------|----------------------------------------------------------------------------|-----------|
| <b>2 0</b> | <b>Comparaison de floats correctement</b>                                  | <b>13</b> |
| <b>2 1</b> | <b>type() vs isinstance()</b>                                              | <b>13</b> |
| <b>2 2</b> | <b>Mini-exercices de production (<i>nouveau</i>)</b>                       | <b>13</b> |
|            | Exercice 1 — Prédiction . . . . .                                          | 13        |
|            | Exercice 2 — Production . . . . .                                          | 13        |
|            | Exercice 3 — Débogage . . . . .                                            | 14        |
|            | <b>Synthèse</b>                                                            | <b>15</b> |
| <b>20</b>  | <b>Questions intelligentes — Les Types en Python</b>                       | <b>15</b> |
| <b>1)</b>  | <b>Pourquoi True + True retourne 2 ?</b>                                   | <b>15</b> |
|            | Énoncé . . . . .                                                           | 15        |
|            | Solution . . . . .                                                         | 15        |
| <b>2)</b>  | <b>Différence entre == et is ?</b>                                         | <b>16</b> |
|            | Énoncé . . . . .                                                           | 16        |
|            | Solution . . . . .                                                         | 16        |
| <b>3)</b>  | <b>Pourquoi deux petites chaînes peuvent partager la même identité ?</b>   | <b>16</b> |
|            | Énoncé . . . . .                                                           | 16        |
|            | Solution . . . . .                                                         | 16        |
| <b>4)</b>  | <b>Pourquoi 256 est parfois identique mais 1000 non ?</b>                  | <b>17</b> |
|            | Énoncé . . . . .                                                           | 17        |
|            | Solution . . . . .                                                         | 17        |
| <b>5)</b>  | <b>Pourquoi une liste modifiée dans une fonction affecte l'extérieur ?</b> | <b>17</b> |
|            | Énoncé . . . . .                                                           | 17        |
|            | Solution . . . . .                                                         | 17        |
| <b>6)</b>  | <b>Pourquoi une valeur par défaut mutable est dangereuse ?</b>             | <b>18</b> |
|            | Énoncé . . . . .                                                           | 18        |
|            | Solution . . . . .                                                         | 18        |
| <b>7)</b>  | <b>Pourquoi tuple contenant une liste reste mutable ?</b>                  | <b>18</b> |
|            | Énoncé . . . . .                                                           | 18        |
|            | Solution . . . . .                                                         | 19        |
| <b>8)</b>  | <b>Pourquoi float("3.14") fonctionne mais float("3,14") non ?</b>          | <b>19</b> |
|            | Énoncé . . . . .                                                           | 19        |
|            | Solution . . . . .                                                         | 19        |
| <b>9)</b>  | <b>Pourquoi 0.1 + 0.2 == 0.3 est faux ?</b>                                | <b>19</b> |

|                                                                                           |           |
|-------------------------------------------------------------------------------------------|-----------|
| Énoncé . . . . .                                                                          | 19        |
| Solution . . . . .                                                                        | 20        |
| <b>10) Pourquoi <code>set([1,2,2,3])</code> supprime les doublons ?</b>                   | <b>20</b> |
| Énoncé . . . . .                                                                          | 20        |
| Solution . . . . .                                                                        | 20        |
| <b>11) Pourquoi une clé dict doit être immuable ?</b>                                     | <b>20</b> |
| Énoncé . . . . .                                                                          | 20        |
| Solution . . . . .                                                                        | 20        |
| <b>12) Différence entre <code>list()</code> et <code>copy()</code></b>                    | <b>21</b> |
| Énoncé . . . . .                                                                          | 21        |
| Solution . . . . .                                                                        | 21        |
| <b>13) Pourquoi <code>deep copy</code> est parfois nécessaire ?</b>                       | <b>21</b> |
| Énoncé . . . . .                                                                          | 21        |
| Solution . . . . .                                                                        | 22        |
| <b>14) Pourquoi <code>isinstance</code> est préférable à <code>type</code> ?</b>          | <b>22</b> |
| Énoncé . . . . .                                                                          | 22        |
| Solution . . . . .                                                                        | 22        |
| <b>15) Pourquoi <code>range</code> n'est pas une liste ?</b>                              | <b>23</b> |
| Énoncé . . . . .                                                                          | 23        |
| Solution . . . . .                                                                        | 23        |
| <b>16) Pourquoi <code>str</code> est immuable ?</b>                                       | <b>23</b> |
| Énoncé . . . . .                                                                          | 23        |
| Solution . . . . .                                                                        | 23        |
| <b>17) Pourquoi dict conserve l'ordre depuis Python 3.7 ?</b>                             | <b>24</b> |
| Énoncé . . . . .                                                                          | 24        |
| Solution . . . . .                                                                        | 24        |
| <b>18) Pourquoi <code>set</code> est non ordonné ?</b>                                    | <b>24</b> |
| Énoncé . . . . .                                                                          | 24        |
| Solution . . . . .                                                                        | 24        |
| <b>19) Pourquoi <code>int("10")</code> fonctionne mais <code>int("10.5")</code> non ?</b> | <b>24</b> |
| Énoncé . . . . .                                                                          | 24        |
| Solution . . . . .                                                                        | 25        |
| <b>20) Cas réel : bug d'égalité sur liste imbriquée</b>                                   | <b>25</b> |
| Énoncé . . . . .                                                                          | 25        |
| Solution . . . . .                                                                        | 25        |

# Les Types en Python

Le support est une propriété de **Bechir Bejaoui**

---

## 1 Pourquoi comprendre les types ?

Un programme robuste repose sur :

- la maîtrise des types manipulés
- la compréhension des conversions implicites
- l'anticipation des erreurs de typage
- la distinction entre mutabilité et immutabilité

En production, la majorité des bugs proviennent d'une mauvaise gestion des types.

---

## 2 Les grandes familles de types

Python distingue principalement :

### Types numériques

- int
- float
- complex

### Type booléen

- bool

### Types séquentiels

- str
- list
- tuple
- range

## Types de collection

- dict
  - set
- 

## 3 Les types numériques

### 3.1 Entiers — int

Caractéristiques :

- précision arbitraire
- pas de limite 32 ou 64 bits
- stockage dynamique

```
"""
Les entiers en Python ont une précision illimitée.
La seule limite est la mémoire disponible.
"""
```

```
x = 10
grand = 10**100

print(type(x))
print(grand)
```

---

### 3.2 Flottants — float

Caractéristiques :

- norme IEEE 754
- précision limitée
- erreurs d'arrondi possibles

```
"""
Les floats sont représentés en binaire.
Certaines valeurs décimales ne sont pas exactes.
"""
```

```
a = 0.1
b = 0.2
```

```
print(a + b)
print((a + b) == 0.3)
```

---

### 3.3 Complex — complex

Structure :  $a + bj$

```
"""
Les nombres complexes possèdent :
- une partie réelle
- une partie imaginaire
Approfondissement en Partie II - Opérateurs avancés.
"""
```

```
z = 2 + 3j
```

```
print(type(z))
print(z.real)
print(z.imag)
```

---

## 4 Bool — un cas particulier

bool hérite de int.

```
"""
True vaut 1
False vaut 0
bool est une sous-classe de int
"""
```

```
print(True + True)
print(isinstance(True, int))
```

---

## 5 Les chaînes — str

Caractéristiques :

- immuables
- indexables
- itérables

```

"""
Les chaînes sont immuables.
Toute modification crée un nouvel objet.
"""

nom = "Python"

print(nom[0])
print(len(nom))
print(type(nom))

```

---

## 6 List — mutable

```

"""
Les listes sont mutables.
On peut modifier leur contenu.
"""

liste = [1, 2, 3]

liste.append(4)
print(liste)

```

---

## 7 Tuple — immutable

```

"""
Les tuples sont immuables.
Plus sûrs et souvent plus performants.
"""

t = (1, 2, 3)
print(t)

Tentative de modification :

"""
Une modification provoque une erreur.
"""

try:
    t[0] = 10
except TypeError as e:

```

```
print("Erreur :", e)
```

---

## 8 Dictionnaire — dict

Structure clé / valeur.

```
"""
Un dictionnaire stocke des paires clé:valeur.
Les clés doivent être immuables.
"""
```

```
personne = {
    "nom": "Alice",
    "age": 30
}

print(personne["nom"])
```

---

## 9 Set — ensemble unique

```
"""
Un set ne contient pas de doublons.
Les éléments doivent être immuables.
"""
```

```
s = {1, 2, 3, 3}
print(s)
```

---

## Range — séquence paresseuse

Caractéristiques :

- ne stocke pas les éléments en mémoire
- génère les valeurs à la demande
- idéal pour les boucles sur de grandes plages

```
"""
range est un objet itérable paresseux.
Il ne stocke pas tous les éléments.
La mémoire utilisée est identique pour range(10) et range(10_000_000).
"""
```

```

"""

r = range(5)

print(type(r))
print(list(r))

# Mémoire identique quelle que soit la taille
grand = range(10_000_000)
print(type(grand))

```

---

## 1 1 Mutabilité vs Immutabilité

| Type    | Mutable ? |
|---------|-----------|
| int     | Non       |
| float   | Non       |
| complex | Non       |
| bool    | Non       |
| str     | Non       |
| tuple   | Non       |
| range   | Non       |
| list    | Oui       |
| dict    | Oui       |
| set     | Oui       |

---

## 1 2 Identité vs Égalité

```

"""
== compare les valeurs
is compare les identités mémoire
"""

a = [1, 2]
b = [1, 2]

print(a == b)
print(a is b)

```

---

## 1 3 Conversion de types

```
"""
Conversions explicites
"""

print(int("10"))
print(float("3.14"))
print(str(100))
```

---

## 1 4 Typage dynamique

Python est dynamiquement typé.

```
"""
Une variable peut changer de type.
"""

x = 10
print(type(x))

x = "Python"
print(type(x))
```

---

## 1 5 Typage fort

Python est dynamiquement typé mais fortement typé.

```
"""
On ne peut pas mélanger types incompatibles.
"""

try:
    print("2" + 2)
except TypeError as e:
    print("Erreur :", e)
```

---

## 1 6 Annotations de type (bonne pratique moderne)

```
"""
Les annotations améliorent la lisibilité
et permettent aux outils statiques d'analyser le code.
"""

def addition(a: int, b: int) -> int:
    return a + b

print(addition(2, 3))
```

---

## 1 7 Vérification de type

```
"""
Utiliser isinstance plutôt que type().
"""

x = 5
print(isinstance(x, int))
```

---

## 1 8 Copie superficielle et copie profonde

Caractéristiques :

- `copy()` duplique la structure externe uniquement
- `deepcopy()` copie récursivement tous les sous-objets
- risque fréquent avec les structures imbriquées

```
"""
copy() copie la structure externe.
Les sous-objets restent partagés.
"""
```

```
import copy

a = [[1, 2], [3, 4]]
b = a.copy()

b[0].append(99)
```

```

print("Original :", a)    # affecté
print("Copie      :", b)

Correction avec deepcopy :

"""
deepcopy() copie tous les niveaux.
L'original n'est plus affecté.
"""

import copy

a = [[1, 2], [3, 4]]
b = copy.deepcopy(a)

b[0].append(99)

print("Original :", a)    # non affecté
print("Copie      :", b)

```

---

## 1 9 Cas réel : bug de mutabilité

```

"""
Attention aux valeurs par défaut mutables.
"""

def ajouter(element, liste=[]):
    liste.append(element)
    return liste

print(ajouter(1))
print(ajouter(2))  # comportement inattendu

Correction :

def ajouter(element, liste=None):
    if liste is None:
        liste = []
    liste.append(element)
    return liste

print(ajouter(1))
print(ajouter(2))

```

---

## 2 0 Comparaison de floats correctement

```
"""  
Ne pas comparer floats directement.  
Utiliser math.isclose.  
"""
```

```
import math  
  
print(math.isclose(0.1 + 0.2, 0.3))
```

---

## 2 1 type() vs isinstance()

```
"""  
isinstance accepte les sous-classes.  
"""
```

```
class A: pass  
class B(A): pass  
  
b = B()  
  
print(type(b) == A)  
print(isinstance(b, A))
```

---

## 2 2 Mini-exercices de production (*nouveau*)

### Exercice 1 — Prédiction

Prédire la sortie avant exécution :

```
a = [1, 2]  
b = a  
b.append(3)  
  
print(a)  
print(a is b)
```

---

### Exercice 2 — Production

Écrire un programme qui :

1. Crée une liste de notes [12, 15, 9, 17]
2. Crée une copie indépendante de cette liste
3. Modifie la copie en ajoutant 20
4. Vérifie que l'original est inchangé

*# Votre solution ici*

Solution

```
import copy

notes = [12, 15, 9, 17]
notes_copie = copy.deepcopy(notes)

notes_copie.append(20)

print("Original :", notes)
print("Copie      :", notes_copie)
print("Inchangé  :", notes == [12, 15, 9, 17])
```

---

### Exercice 3 — Débogage

Ce code contient 2 erreurs. Les identifier et les corriger.

```
def historique(valeur, log=[]):
    log.append(valeur)
    return log

print(historique("login"))
print(historique("logout"))  # comportement inattendu
```

Solution

*# Erreur 1 : valeur par défaut mutable → partagée entre appels*  
*# Erreur 2 : pas de protection contre l'accumulation involontaire*

```
def historique(valeur, log=None):
    if log is None:
        log = []
    log.append(valeur)
    return log

print(historique("login"))
print(historique("logout"))
```

---

## Synthèse

- Python est dynamiquement typé et fortement typé
  - La mutabilité est un concept fondamental
  - `range` est un itérable paresseux, pas une liste
  - `copy()` est superficielle — utiliser `deepcopy()` pour les structures imbriquées
  - Les floats doivent être manipulés avec précaution
  - `isinstance` est préférable à `type`
  - Les annotations de type améliorent la qualité du code
- 

## 20 Questions intelligentes — Les Types en Python

### 1) Pourquoi `True + True` retourne `2` ?

#### Énoncé

Expliquer pourquoi l'addition de deux booléens retourne un entier.

#### Solution

```
"""
Explication
bool est une sous-classe de int.
True vaut 1.
False vaut 0.
Donc les opérations arithmétiques fonctionnent.
"""

print(True + True)
print(True * 5)
print(isinstance(True, int))
```

---

## 2) Différence entre == et is ?

### Énoncé

Comparer égalité de valeur et identité mémoire.

### Solution

```
"""
Explication
== compare les valeurs.
is compare les adresses mémoire (identité objet).
"""

a = [1, 2]
b = [1, 2]
c = a

print(a == b) # même contenu
print(a is b) # objets différents
print(a is c) # même objet
```

---

## 3) Pourquoi deux petites chaînes peuvent partager la même identité ?

### Énoncé

Comprendre l'interning des chaînes.

### Solution

```
"""
Explication
Python optimise certaines chaînes courtes (interning).
Elles peuvent partager la même référence mémoire.
"""

a = "hello"
b = "hello"

print(a is b)
```

---

#### 4) Pourquoi 256 est parfois identique mais 1000 non ?

##### Énoncé

Expliquer le cache des petits entiers.

##### Solution

```
"""
Explication
Python met en cache les petits entiers (généralement -5 à 256).
Au-delà, les objets sont distincts.
"""

a = 256
b = 256
print(a is b)

x = 1000
y = 1000
print(x is y)
```

---

#### 5) Pourquoi une liste modifiée dans une fonction affecte l'extérieur ?

##### Énoncé

Comprendre le passage par référence d'objet.

##### Solution

```
"""
Explication
Les objets mutables sont modifiés en place.
La fonction reçoit la référence vers l'objet.
"""

def ajouter_element(liste):
    liste.append(99)

ma_liste = [1, 2, 3]
ajouter_element(ma_liste)
```

```
print(ma_liste)
```

---

## 6) Pourquoi une valeur par défaut mutable est dangereuse ?

### Énoncé

Identifier un bug fréquent.

### Solution

```
"""
Explication
La valeur par défaut est évaluée une seule fois.
Elle est partagée entre les appels.
"""

def ajouter(valeur, liste=[]):
    liste.append(valeur)
    return liste

print(ajouter(1))
print(ajouter(2))  # comportement inattendu
```

Correction :

```
def ajouter(valeur, liste=None):
    if liste is None:
        liste = []
    liste.append(valeur)
    return liste

print(ajouter(1))
print(ajouter(2))
```

---

## 7) Pourquoi tuple contenant une liste reste mutable ?

### Énoncé

Comprendre la mutabilité interne.

## Solution

```
"""
Explication
Le tuple est immuable,
mais il peut contenir un objet mutable.
"""

t = (1, [2, 3])
t[1].append(4)

print(t)
```

---

8) Pourquoi `float("3.14")` fonctionne mais `float("3,14")` non ?

### Énoncé

Comprendre le parsing numérique.

## Solution

```
"""
Explication
Python utilise le point comme séparateur décimal.
La virgule provoque une erreur.
"""

print(float("3.14"))

try:
    print(float("3,14"))
except ValueError as e:
    print("Erreur :", e)
```

---

9) Pourquoi `0.1 + 0.2 == 0.3` est faux ?

### Énoncé

Comparer correctement des floats.

## Solution

```
"""
Explication
Les floats sont approximatifs.
Utiliser math.isclose.
"""
```

```
import math

a = 0.1 + 0.2
print(a == 0.3)
print(math.isclose(a, 0.3))
```

---

## 10) Pourquoi set([1,2,2,3]) supprime les doublons ?

### Énoncé

Comprendre la structure d'un set.

### Solution

```
"""
Explication
Un set ne conserve que des éléments uniques.
Il est basé sur le hachage.
"""
```

```
s = set([1, 2, 2, 3])
print(s)
```

---

## 11) Pourquoi une clé dict doit être immuable ?

### Énoncé

Expliquer la contrainte des clés.

### Solution

```
"""
Explication
```

```
Les clés sont basées sur un hash.  
Un objet mutable changerait son hash.  
"""
```

```
d = {"a": 1}
```

```
try:  
    d[[1, 2]] = 3  
except TypeError as e:  
    print("Erreur :", e)
```

---

## 12) Différence entre list() et copy()

### Énoncé

Comparer copie superficielle.

### Solution

```
"""  
Explication  
Les deux créent une copie superficielle.  
Les sous-objets restent partagés.  
"""
```

```
import copy  
  
a = [[1, 2]]  
b = list(a)  
c = copy.copy(a)  
  
b[0].append(99)  
  
print(a)  
print(c)
```

---

## 13) Pourquoi deep copy est parfois nécessaire ?

### Énoncé

Démontrer l'intérêt de deepcopy.

## Solution

```
"""
Explication
deepcopy copie récursivement tous les sous-objets.
"""

import copy

a = [[1, 2]]
b = copy.deepcopy(a)

b[0].append(99)

print(a)
print(b)
```

---

## 14) Pourquoi isinstance est préférable à type ?

### Énoncé

Comprendre le polymorphisme.

## Solution

```
"""
Explication
isinstance accepte les sous-classes.
type vérifie strictement le type exact.
"""

class A: pass
class B(A): pass

b = B()

print(type(b) == A)
print(isinstance(b, A))
```

---

## 15) Pourquoi range n'est pas une liste ?

### Énoncé

Comprendre l'optimisation mémoire.

### Solution

```
"""
Explication
range est un objet itérable paresseux.
Il ne stocke pas tous les éléments.
"""

r = range(1000000)
print(type(r))
```

---

## 16) Pourquoi str est immuable ?

### Énoncé

Montrer l'impossibilité de modification.

### Solution

```
"""
Explication
Les chaînes sont immuables.
Toute modification crée un nouvel objet.
"""

s = "Python"

try:
    s[0] = "J"
except TypeError as e:
    print("Erreur :", e)
```

---

## 17) Pourquoi dict conserve l'ordre depuis Python 3.7 ?

### Énoncé

Expliquer le comportement moderne.

### Solution

```
"""
Explication
Depuis Python 3.7, l'ordre d'insertion est garanti.
"""
```

```
d = {"a": 1, "b": 2, "c": 3}
print(d)
```

---

## 18) Pourquoi set est non ordonné ?

### Énoncé

Observer comportement.

### Solution

```
"""
Explication
Un set est basé sur une table de hachage.
L'ordre n'est pas garanti.
"""
```

```
s = {5, 1, 3}
print(s)
```

---

## 19) Pourquoi int("10") fonctionne mais int("10.5") non ?

### Énoncé

Comprendre les conversions.

## Solution

```
"""
Explication
int attend une représentation entière.
Pour un float sous forme texte, convertir d'abord en float.
"""

print(int("10"))

try:
    print(int("10.5"))
except ValueError as e:
    print("Erreur :", e)

print(int(float("10.5")))
```

---

## 20) Cas réel : bug d'égalité sur liste imbriquée

### Énoncé

Pourquoi la modification d'une copie superficielle affecte l'original ?

### Solution

```
"""
Explication
Une copie superficielle duplique seulement la structure externe.
Les objets internes restent partagés.
"""

a = [[1, 2], [3, 4]]
b = a.copy()

b[0].append(99)

print("Original :", a)
print("Copie :", b)
print("Même sous-liste :", a[0] is b[0])
```